

オープンソースの文化(主題1.11)

1. **开源社区 (Open Source Community)**: 指的是开发者、用户以及志愿者们为了共同改进、维护某个开源软件 (OSS)，并在其基础上进行协作、讨论和资源共享而自发形成的集合。

核心活动: 共同开发 (编码)、提交缺陷报告 (Bug Report)、编写文档以及技术问答。

✅ 正确答案

OSSの共同開発や情報交換を行う人々の集まり (进行 OSS 共同开发和情报交换的人们的集合)

🔍 选项解析与详细说明

1. 为什么选第一个?

开源社区的本质是“人”的集合。它不仅仅是一个技术平台，更是一种基于“自由”和“贡献”的协作文化。通过 GitHub、邮件列表或论坛，全球的开发者可以跨越地域合作。

❌ 错误选项解析

- **Webページ上に記事や... (论坛/BBS)**: 这只是情报交换的一种“手段”或“工具”，并不是社区本身的定义。
- **電子メールを複数の人に... (邮件列表)**: 这描述的是 **Mailing List (メーリングリスト)**，是社区沟通的工具之一。
- **インターネット上でリアルタイムに... (即时通讯/聊天)**: 这描述的是 **Chat (チャット)** 或 **IRC**。
- **ソフトウェアの利用者が遵守すべき... (许可协议)**: 这描述的是 **License (ライセンス)**，即开源许可证。

2. **开源社区 (オープンソースコミュニティ)** 的运作模式。开源社区的核心在于**自发性、公开性以及异步协作**。

💡 知识点总结

- **开源社区的特征**: 由全球各地的志愿者组成，主要通过互联网进行**非营利性**的协作。
- **常用交流手段**: 为了让信息透明且可存档，社区通常使用异步（非实时）或公开的在线工具。

✅ 正确答案

コールセンター (呼叫中心 / 客服中心)

🔍 选项解析与详细说明

为什么“呼叫中心”是不合适的?

- **商业 vs. 社区**: 呼叫中心通常是**商业公司**为付费客户提供的服务，需要雇佣专门的客服人员。

- **成本与规模**：开源社区通常没有资金去维持一个 24 小时的人工热线。
- **信息共享**：呼叫中心是一对一的私下交流，信息无法被社区其他成员检索和共享。

✗ 其他选项为什么是合适的？

- **掲示板・フォーラム (论坛/看板)**：最常用的手段，方便大家发帖提问并留下记录供后人参考。
- **チャット (即时聊天)**：如 Slack、Discord 或 IRC，用于实时讨论技术问题。
- **メーリングリスト (邮件列表)**：传统且核心的交流方式，很多大型项目（如 Linux 内核）的开发讨论都在这里进行。
- **各地でのイベント (线下活动)**：如开源年会 (OSC) 或黑客松 (Hackathon)，用于面对面的技术交流和社交。

3. 自由软件运动的历史及其核心组织。

💡 知识点总结

- **GNUプロジェクト (GNU Project)**：由 **Richard Stallman (リチャード・ストールマン)** 于 1983 年发起，目标是开发一个完全自由的操作系统——**GNU 操作系统**。
- **核心理念**：软件应该是自由的，用户应该拥有运行、复制、分发、研究和修改软件的自由。

✅ 正确答案

フリー（自由）なオペレーティングシステムを開発する目的で、リチャード・ストールマンによって立ち上げられたオープンソースコミュニティはGNUプロジェクト

🔍 选项解析与详细说明

为什么选 GNUプロジェクト？

Richard Stallman 发起这个项目的初衷是因为当时商业操作系统的闭源限制，他希望建立一个兼容 Unix 但完全自由的系统。

- **成果**：开发了编译器 (GCC)、编辑器 (Emacs) 等许多核心工具。
- **关系**：现在的“Linux 系统”实际上大部分是由 GNU 工具组成的，因此常被称为 **GNU/Linux**。

✗ 为什么其他选项不对？

- **Linux Foundation**：成立于 2000 年，旨在促进 Linux 的增长和标准化。它不是由 Stallman 创立的，Stallman 更倾向于使用“自由软件”一词而非单纯的“Linux”。
- **Open Source Initiative (OSI)**：1998 年成立，提出了“开源 (Open Source)”的概念。Stallman 对此持保留意见，因为他认为“开源”一词淡化了“自由 (Freedom)”的重要性。
- **Free Software Foundation (FSF)**：虽然这也是 Stallman 创立的组织，但它是作为 GNU 项目的**法律和财务支持机构**成立的。题目问的是为了“开发操作系统”而发起的**社区项目**，最贴切的答案是 **GNU 项目**。

4. **生态系统 (エコシステム / Ecosystem)** 在开源领域的具体含义。在 IT 行业中，这个词通常用来描述一个多方参与、互惠共赢的“循环圈”。

 知识点总结

- **生态系统 (Ecosystem)**: 原本是生物学词汇。在 OSS 领域，它指 **开发者、用户、社区和企业** 之间形成的一种共生关系。
- **核心逻辑**: 企业提供资源或资金，开发者贡献代码，用户使用并提供反馈，这种循环推动了软件的持续进化和商业价值的实现。

 正确答案

開発者/ユーザコミュニティや企業が連携しながらOSSを開発・発展させていく循環のこと (开发者/用户社区与企业相互协作，共同开发并促进 OSS 发展的循环)

 选项解析与详细说明

1. 为什么“循环 (循環)”是关键词？

生态系统强调的是一种**自我持续的生命力**。

- **开发者**: 通过贡献代码获得技术提升和名望。
- **企业**: 通过集成 OSS 降低成本，并回馈社区以确保软件的稳定。
- **用户**: 通过使用软件发现 Bug 并提出需求。这种多方的“联动”让 OSS 不仅仅是一个静态的代码库，而是一个动态发展的生命体。

 错误选项解析

- **OSSの開発や改善に... (人々が集まり)**: 这描述的是 **开源社区 (Community)**。
- **ソフトウェアの利用者が遵守... (文書)**: 这描述的是 **开源许可证 (License)**。
- **コンピュータソフトウェアのフリー... (団体)**: 这通常指 **FSF** 等推进团体。
- **オープンソースソフトウェア...のこと**: 这只是对 **OSS/FS 软件本身**的简称，不是生态系统。

5. 开源协议在**云计算和网络服务时代**的演进。这是一个非常经典且具有现代意义的 LPI 考点。

 知识点总结

- **コピーレフト (Copyleft)**: 指如果修改并发布了软件，则必须公开源代码的机制。
- **ASP 漏洞 (ASP Loophole)**: 传统的 GPL 协议规定，只有在“分发 (配布)”软件时才需要公开源代码。如果程序只在服务器上运行，通过互联网提供服务 (如 SaaS)，而不把程序发给用户，就可以不公开源代码。
- **AGPL (Affero GPL)**: 专门为了堵住这个漏洞而设计的协议。它规定即便只是通过网络提供服务，也视同为“发布”，必须应要求公开源代码

 正确答案

AGPL (GNU Affero General Public License)

选项解析与详细说明

为什么是 AGPL?

AGPL 是在 GPLv3 的基础上增加了一个“远程网络交互”条款。

- **核心逻辑**：只要用户通过网络（Internet）与程序交互，即使你没有把安装包发给用户，你也必须向这些用户提供源代码。
- **适用场景**：云服务、在线协作工具、基于 Web 的管理系统等。

其他选项解析

- **GPL (General Public License)**：最著名的强传染性协议。但它存在“ASP 漏洞”，即不分发程序包就不用给源码。
- **LGPL (Lesser GPL)**：相对宽松的 GPL。主要用于类库（Library），允许闭源程序动态链接而不受传染。
- **BSD License**：极度宽松的协议。不要求公开源码，可以随意闭源商业化。
- **IGPL**：这是一个虚构的干扰项，不存在这个主流协议。

6. 开源协议中关于**“传染性范围”的界定，特别是在类库（Library）**使用场景下的特殊规定。

知识点总结

- **LGPL (Lesser GPL)**：原本叫 Library GPL，是专门为**类库**设计的协议。
- **核心逻辑**：如果你的程序只是**调用（链接）了 LGPL 协议的库**，而**没有修改库本身的源代码**，那么**你的程序不需要开源**。
- **权利范围**：LGPL 的约束力仅限于该库本身，不会“传染”给调用它的主程序。

正确答案

LGPL

选项解析与详细说明

a. 为什么选 LGPL?

GPL 协议具有很强的“传染性”，即只要你的程序连接了 GPL 库，整个程序都必须变成 GPL 并开源。这对于商业软件开发者来说非常困难。

为了让开源库被更广泛地采用，**LGPL** 诞生了：

- 它允许商业闭源软件通过**动态链接**的方式使用该库。
- 这种特性被称为“**弱传染性**”。

其他选项解析

- **GPL (General Public License)**：具有强传染性。如果你的程序使用了 GPL 代码，整个软件的权利都会受到 GPL 约束，必须开源。

- **AGPL (Affero GPL)**: 比 GPL 更严格, 专门针对网络服务。只要通过网络提供功能, 调用者就受约束。
- **FLOSS**: 这是 Free/Libre and Open Source Software 的缩写, 是“自由/开源软件”的总称, 不是具体的协议名。
- **フリーソフトウェア (Free Software)**: 指“自由软件”这一类别, 也不是具体的协议。

考试小秘籍

- 关键词匹配: 题目中出现「ライブラリ用」(类库用) 或「別のソフトウェアまで権利が及ばない」(权利不波及其他软件), 直接秒选 LGPL。
 - 区分“强”与“弱” :
 - GPL = 强传染 (全家都要开源)。
 - LGPL = 弱传染 (库开源即可, 主程序随意)。
 - 注意单词缩写: 在 LPI 考试中, GPL 系列 (GPL, LGPL, AGPL) 是协议题的三大支柱, 一定要分清它们的应用场景。
7. 著作物に対して著作権や特許権などの知的財産権が発生していない状態、または失効した状態のことを「パブリックドメイン (公有)」といいます。

法律和知识产权方面的术语。在 IT 和软件领域, 了解作品处于什么“法律状态”非常重要。

知识点总结

- **パブリックドメイン (Public Domain)**: 指作品的知识产权 (著作权、专利权等) 已经**过期**、**被放弃**或**从一开始就不适用**的状态。
- **核心特征**: 处于这种状态的作品, 任何人都可以自由地使用、修改、再分发, 甚至可以将其用于商业目的, 而无需经过任何人的许可。

正确答案

パブリックドメイン

(公有领域)

考试小秘籍

- 关键词匹配: 看到「失効」、「発生していない」、「知的財産権がない」, 直接联想 **Public Domain**。
 - 区分“免费”与“无权” :
 - “免费”不代表没有著作权。
 - 只有“公有领域”才是彻底放弃或丧失了权利。
 - 注意 **CC0** 标识: 在现代, 如果作者想把作品变成 Public Domain, 通常会使用 **CC0 (Creative Commons Zero)** 宣言。
8. 这道题考查的是开源协议中最重要的特性之一: **传染性 (Viral Nature)**。

在 LPI 考试中，你需要区分哪些协议是“霸道”的（必须一起开源），哪些是“温柔”的（允许商业闭源结合）。

💡 知识点总结

- **コピーレフト (Copyleft / 传染性)**：指的是当你把 OSS 代码和自己的代码结合并再分发时，你的代码也必须强制开源。
- **非传染性/弱传染性**：指协议允许你的私有代码保持闭源，只要求你对 OSS 代码本身的修改进行开源（或者完全不要求）。

✅ 正确答案

- **MPL (Mozilla Public License)**
- **LGPL (GNU Lesser General Public License)**
- **BSD License**

🔍 选项解析与详细说明

a. 为什么这三个不需要公开其他代码？

- **BSD License**：这是最“大方”的协议。它几乎没有任何限制，你可以把它的代码拿来，关起门来做成商业软件卖掉，完全不需要公开你的任何源代码。
- **LGPL**：专门为类库设计。它允许你的程序通过“链接”的方式使用它，而你的主程序代码可以保持闭源（**弱传染性**）。
- **MPL**：它的传染性是**基于文件**的。你修改了 MPL 协议的文件，那个文件要开源；但你新写的、独立的其他文件，可以不公开。

❌ 为什么 GPL 和 AGPL 不行？

- **GPL**：典型的“强传染性”。只要你的代码里用了一丁点 GPL 代码，整个“作品”都被视为派生作品，必须全部开源。
- **AGPL**：比 GPL 更狠。即使不分发安装包，只要在云端跑服务，全家都要开源。

口诀：小妹不传染

（解释：**LGPL / MPL / BSD**，这三个“小妹”性格温柔，不传染别人）

- **小(L)**：代表 **LGPL**（L 是 Lesser，也是 Library，是个“小”号的 GPL）。
- **妹(M)**：代表 **MPL**（Mozilla 的首字母 M，谐音“妹”）。
- **不(B)**：代表 **BSD**（首字母 B）。
- **传(C)**：代表 **Copyleft**（传染性）。

🔍 选项解析与详细说明

为了让你在考场上秒杀这类题，我们把这三个“小妹”的特征再精炼一下：

- i. **BSD（不讲理的大方）**：

- “你拿去用吧，改名换姓卖了都行，别提我就行。”

ii. LGPL（类库的温柔）：

- “只要你只是‘调用’我，不改动我的身体，你的代码依然是你的，不用开源。”

iii. MPL（文件级的隔离）：

- “你改了我的文件，那个文件要开源；但你自己新建的文件，那是你的隐私。”

日语对应：

- 及ばない（波及不到） \rightarrow 选 LGPL/BSD。

- 求めない（不要求） \rightarrow 选 LGPL/BSD/MPL。

9. 这道题考查的是 **强传染性（Strong Copyleft）** 协议的核心特征。也就是无论你是否修改了代码，只要你发布了软件，就必须公开源代码。

💡 知识点总结

- **コピーレフト（Copyleft）**：这种机制利用著作权法，强制要求作品的派生版本必须保持与原版本相同的自由度。
- **ソースコードの公開義務**：
- **GPL 系列**：属于“强传染性”，要求最为严格。
- **宽松型协议（MIT/BSD/Apache）**：属于“非传染性”，允许闭源商业化。

✅ 正确答案

- **GPL（GNU General Public License）**
- **LGPL（GNU Lesser General Public License）**

注意：题目问的是“源代码公开”。虽然 LGPL 在库调用上比较宽松，但如果你**再发布（再配布）**这个软件（包含这个库），你仍然必须公开该 LGPL 库部分的源码（即使你没改过它）。而 GPL 则是要求整个软件都要公开。

❌ 为什么其他选项不对？

- **MIT / BSD / Apache**：这三剑客是**商业最友好**的。
- 特征：它们只要求你保留原作者的版权声明（著作権表示），完全**不要求**你公开源代码。
- 你可以把它们的代码拿走，改吧改吧，然后直接卖闭源的安装包。

关键词匹配：

- 「公開が必要」（需要公开） \rightarrow 找 **GPL 家族**。

- 「表示のみ / 公開不要」（仅声明/不需公开） \rightarrow 找 **MIT/BSD/Apache**。

10. OSS（开源软件）的核心精神

- **OSS != 没版权**：OSS 虽然可以自由改，但作者依然拥有**著作权（Copyright）**。你不能删掉原作者的名字。

- **无担保 (No Warranty)**: 这是为了保护开发者。既然免费给你用，如果你拿它跑业务出了 Bug 亏了钱，开发者是不负责赔偿的。

- **OSI 的角色**: OSI 像是一个“裁判”，只有符合它制定的 **OSD (Open Source Definition)** 规则的协议，才能被称为真正的“开源协议”。

11. プログラムやソースコードに対して、秘密保持契約の締結や利用料の支払いが必要など、利用に関する制限がかけられているものを「**プロプライエタリ（私有の）ソフトウェア**」といいます。Microsoft WindowsやOracle Databaseなどがこれにあたります。

12. 在 LPI 考试中，**Copyleft（左版/复制左）** 是区分不同开源协议（如 GPL vs BSD）的分水岭。

💡 知识点总结

- **Copyleft 的本质**: 它并不废除著作权 (Copyright)，而是**利用**著作权法来“强制”保证软件永远自由。

- **核心逻辑**: 你可以自由使用和改动，但你**必须**以同样的自由度把改动后的作品传下去。这就是所谓的“传染性”。

- **对比记忆**:

- **Copyright (版权)**: 目的是“**禁止**”他人自由使用（所有权私有）。

- **Copyleft (左版)**: 目的是“**保证**”他人自由使用（所有权共有，但必须保持自由）。

✅ Copyleft 的四大支柱

1. **自由分发**: 不限制你把软件给别人。

· 派生物（著作物を改変したもの）についても利用やコピー・再配布、改変を制限しない

2. **强制开源**: 分发时必须能让人看到“设计图”（源码）。

コピー・再配布の際はソースコードなど全ての情報を入手可能とする

3. **继承性**: 你改了之后，不能把它变成闭源软件卖，必须继续沿用 Copyleft 协议。

4. **互惠性**: 如果你从社区拿走了自由的代码，你贡献回来的也必须是自由的。

- 同じライセンスを著作物および派生物にも適用し、明記する

つまりコピーレフト (copyleft) とは、著作権 (copyright) の基本的な考え方『この著作物は会社/個人に帰属するので他者の勝手な利用や模倣を禁ずる』とは対照的な考え方をいいます。

- 派生物（著作物を改変したもの）についても利用やコピー・再配布、改変を制限しない

13. 这道题考查的是 自由软件精神的核心协议。题目中提到的“自由软件基金会 (FSF)”、“Copyleft 根源”以及“对派生物严格规定”，全都在指向同一个答案。

💡 知识点总结

- **GPL (GNU General Public License)**: 这是 Copyleft 阵营的“带头大哥”，由 Richard Stallman 为 GNU 项目量身打造。

· 强传染性 (Strong Copyleft) : GPL 规定, 如果你在程序中使用了 GPL 的代码, 那么你的整个程序 (派生物) 在发布时, 也必须采用 GPL 协议并公开源代码。这就是所谓的“严格规定”。

✅ 正确答案

GPL

🔍 选项解析与详细说明

a. 为什么选 GPL?

GPL 的核心目标是“保护自由”。为了防止有人把自由的代码拿走后闭源变成私有财产, 它设立了严格的继承规则:

- 改动后: 必须开源。
- 发布时: 必须附带源码。
- 协议: 必须继续用 GPL。

❌ 其他选项解析 (非 Copyleft 或 弱 Copyleft)

- MIT / BSD / Apache License: 这三者属于 Permissive (许可型/宽松型)。它们对派生物几乎没有限制, 允许你拿去改了以后闭源卖钱。它们不符合“严格规定派生物”的特征。
- MPL (Mozilla Public License): 虽然也是 Copyleft, 但它是弱 Copyleft (基于文件级别)。比起 FSF 亲生的 GPL, 它的规定要宽松得多。

口诀: FSF 亲儿子, GPL 最严格, 只要占了边, 源码必须给。

📖 考试小秘籍

1. 关键词锁定:

- 看到 **FSF + リチャード・ストールマン** \rightarrow 选 **GPL**。
- 看到 **派生物についても厳しく規定** \rightarrow 选 **GPL**。

2. 分类排除:

- **GPL / LGPL** = Copyleft 派 (要源码)。
- **MIT / BSD / Apache** = Permissive 派 (不要源码)。

14. Mozilla Foundationによって作成されたオープンソースライセンスで、このライセンスのソフトウェアはソースコードを公開する必要があるが、他のライセンスのソフトウェアやプロプライエタリなソフトウェアと組み合わせても、組み合わせたソフトウェアのソースコードの公開を必要とせずに再配布できるものは**MPL**

由Mozilla基金会创建的开源许可证, 使用此许可证的软件必须公开源代码, 但与其他许可证的软件或专有软件组合时, 即使不公开组合后软件的源代码也可重新分发的许可证就是MPL。

15. マサチューセッツ工科大学で作成されたBSD系のオープンソースライセンスで、GPLと比べて規制が少ないものは**MIT License**

由麻省理工学院制定的BSD系开源许可证中，与GPL相比限制较少的是MIT许可证。

16. **Apache License**はApacheソフトウェア財団（ASF）によって作成された、BSD系のライセンスです。Apache License 2.0にはGPLv3と同じく特許の停止条件が入っているため、両者は互換性があるとされています。

Apache许可证是由Apache软件基金会（ASF）制定的BSD类许可证。由于Apache许可证2.0与GPLv3一样包含专利终止条款，因此两者被认为是兼容的。

17. 这道题考查的是 **GPL 协议演进史** 中的一个重大转折点。虽然 GPLv2 和 GPLv3 在“自由软件”的核心哲学上是一致的，但为了应对现代技术和法律的挑战，v3 做了针对性的升级。

💡 知识点总结

GPLv3 相比 v2，最重要的改进在于处理**现代法律和硬件限制**：

- **特許（特許 / Patent）**：增加了防止恶意专利诉讼的条款。
- **TiVo化（Tivoization）**：禁止硬件厂商通过硬件手段限制用户修改并运行自己的代码。
- **国际化**：使用了更通用的法律术语，使其在世界各国更易执行。
- **为什么“特許”是核心区别？**

在 GPLv2 时代，开源软件经常面临“专利陷阱”。

- **v3 的改进**：GPLv3 明确规定，任何分发软件的人，如果拥有该软件中的某项专利，就必须**授权**给所有用户免费使用。
- **互不侵犯**：如果某人利用专利起诉其他 GPLv3 用户，他将自动丧失该软件的所有授权许可。

GPLv2 像是一个**朴素的法律文件**，它只管代码能不能看、能不能改。

GPLv3 像是一个**全副武装的升级版**。它发现现在有人钻法律空子，比如：

- 虽然代码公开了，但申请了**专利 (Patent)**，你一用我就告你。
- 代码虽然能改，但**硬件设备 (TiVo)** 锁死了，你改了也装不进去。

所以 **v3 专门补上了这些漏洞**。

18. BSD（Berkeley Software Distribution）ライセンスはカリフォルニア大学によって作成されました。カリフォルニア大学のバークレー校が開発したOSやソフトウェア群に対して採用されています。無保証であることを明記、また著作権とライセンス条文を表示すれば再配布できます。派生物へのライセンス適用は不要です。

BSD（Berkeley Software Distribution）许可证由加利福尼亚大学制定。该许可证被加利福尼亚大学伯克利分校开发的操作系统和软件采用。只要明确声明无保证，并显示版权和许可条款，即可进行再分发。无需将许可证应用于派生作品。

19. ソフトウェア開発において、既存のプロジェクトから新しいプロジェクトが派生して開始されることを「フォーク」といいます。具体的には、既存のプロジェクトのソースコードをコピーし、それに変更を加えて新しいプロジェクトとして進めていくことを指します。特にOSS（オープンソースソフトウェア）の開発では、元の開発者の許可なしに新しいプロジェクトを開始できるため、フォークが起こりやすくなっています。

在软件开发中，从现有项目派生并启动新的项目被称为“分叉”。具体来说，是指复制现有项目的源代码，对其进行修改后作为新的项目推进。特别是在OSS（开源软件）开发中，由于可以在未经原开发者许可的情况下启动新的项目，因此更容易发生分叉。

20. GPLのソフトウェアを外部に再配布する際は、ソースコードを公開する義務があります。これは、ソフトウェアが改変されているか否かにかかわらず適用されます。改変されている場合は、その改変部分のソースコードも公開する必要があります。一方、自組織内での利用や改変については自由であり、ソースコードの公開義務は発生しません。

在对外再分发GPL软件时，有公开源代码的义务。这一规定无论软件是否被修改均适用。如果软件被修改，则还需要公开修改部分的源代码。另一方面，在组织内部使用和修改则不受限制，也不会产生公开源代码的义务。

21. 「FOSS（Free/Open Source Software）」や「FLOSS（Free/Libre and Open Source Software）」は、自由にソースコードの入手、改変が行えるソフトウェアを総称したものです。オープンソースソフトウェア、フリーソフトウェアはFOSS、FLOSSに該当します。

「FOSS（Free/Open Source Software）」和「FLOSS（Free/Libre and Open Source Software）」是对可自由获取和修改源代码的软件的统称。开源软件、自由软件都属于FOSS、FLOSS的范畴。